

Neighbourhood Echoes

Step-by-Step Development Plan

NPC Memory Engine for Games using Cognition.ai

Generated: June 27, 2026

A compact social mystery / life-sim where NPCs remember player actions, promises, secrets, rumours, and contradictions - then change dialogue, trust, and story paths accordingly.

Table of Contents

1. Lock the game vision
2. Define the MVP scope
 - MVP target
 - MVP content
 - MVP success condition
3. Choose the technical architecture
4. Pick the stack
 - MVP stack
5. Design the NPC memory model
6. Design Cognee datasets
7. Build the core memory operations
8. Create the memory importance system
9. Design NPC personalities
10. Build the dialogue pipeline
11. Build authored dialogue fallbacks
12. Build the gossip system
13. Build the relationship system
14. Build the mystery system
15. Design NPC daily simulation
16. Create the backend API
17. Create the memory schema
18. Implement memory retrieval recipes
19. Build the first vertical slice
 - Phase 1: Prototype memory only
 - Phase 2: Game-engine walking prototype
 - Phase 3: Promise system
 - Phase 4: Mystery clue system
 - Phase 5: Gossip system
 - Phase 6: Relationship-driven branching
20. Build debugging tools early
21. Handle latency and cost
22. Memory decay and summarisation
23. Content production plan
24. Build the AI safety and consistency layer
25. Save/load strategy
26. Testing plan
27. Production milestones
 - Milestone 0: Design lock
 - Milestone 1: Cognee memory spike
 - Milestone 2: Dialogue vertical slice
 - Milestone 3: Persistent NPC memory
 - Milestone 4: Promise and consequence system
 - Milestone 5: Mystery gameplay

Milestone 6: Gossip simulation

Milestone 7: Full MVP polish

28. Recommended folder structure

29. First implementation order

30. MVP feature checklist

31. Key design risks

32. Final target experience

Working game definition: Neighbourhood Echoes is a social mystery / life-sim game where every NPC remembers what the player said, did, promised, ignored, revealed, or hid. NPCs also form opinions, spread rumours, compare memories, and change future dialogue/quests based on their personal and shared memory.

The core technical idea should be: **the game engine handles moment-to-moment gameplay, while a Cognee-powered memory service stores, retrieves, enriches, and forgets NPC memories.** Cognee is a good fit because its v1.0 workflow is built around `remember`, `recall`, `improve`, and `forget`, and it stores information through relational, vector, and graph-backed memory systems for provenance, semantic retrieval, and relationship reasoning.

1. Lock the game vision

1.1 Core fantasy

The player moves into a small neighbourhood where everyone knows everyone. Every interaction leaves an "echo": NPCs remember events, interpret them emotionally, and reuse those memories later.

The promise to the player:

People remember what you did, what you said, what you promised, and who heard about it.

1.2 Genre

Best starting genre:

Narrative social sim + neighbourhood mystery

Think of a compact, replayable game rather than an open-world RPG. The first version should focus on a few streets, 8-12 NPCs, and one central mystery.

1.3 Core loop

The player's daily loop:

1. Explore the neighbourhood.
2. Talk to NPCs.
3. Do small favours or make choices.
4. Trigger memories in NPCs.
5. Hear rumours or contradictions.
6. Use remembered information to solve social/mystery problems.
7. End the day.
8. NPC relationships and beliefs update overnight.

The main differentiator is not just dialogue generation. The differentiator is **persistent social consequence**.

2. Define the MVP scope

Do not begin with a large simulation. Build a vertical slice that proves memory-driven gameplay.

MVP target

One playable neighbourhood block over 3 in-game days.

MVP content

Area	MVP scope
Map	1 street, 5-7 explorable locations
NPCs	6 major NPCs, 4 background NPCs
Time	Morning, afternoon, evening
Main story	One mystery: "Who broke into the community garden shed?"
Side stories	3 personal NPC arcs

Area	MVP scope
Memory system	NPC remembers player actions, dialogue, promises, secrets, witnessed events
Gossip system	NPCs can pass selected memories to other NPCs
Relationship system	Trust, suspicion, affection, fear, resentment
Dialogue	Hybrid authored + AI-assisted
Save system	Persistent world state + persistent Cognee memory dataset

MVP success condition

The vertical slice is successful when two players can play the same 3-day mystery and get meaningfully different NPC reactions because of different choices.

3. Choose the technical architecture

Recommended architecture

Use a **game client + backend memory service**.

```

Game Engine
(Unity / Godot / Unreal)
  |
  | HTTP / WebSocket
  v
Game Backend API
FastAPI / Node / C# service
  |
  | Python Cognee wrapper
  v
Cognee Memory Layer
Relational store + Vector store + Graph store
  |
  v
LLM + Embedding Provider

```

Cognee's Python API exposes top-level calls such as `remember()` and `recall()`, and the quickstart shows this as an async workflow. For a game, it is cleaner to keep this in a backend service rather than calling Cognee directly from the game client.

Why backend service instead of direct engine integration?

A backend gives you:

Need	Why backend helps
API key safety	LLM and Cognee keys never ship in the game build
Latency control	Cache, queue, batch, and retry memory operations
Debugging	Inspect memories, prompts, failures, costs
Portability	Unity, Godot, Unreal, web builds can all use the same memory API
Live tuning	Adjust prompts, retrieval strategy, and memory schema without rebuilding the client

4. Pick the stack

MVP stack

Layer	Recommended choice
Game engine	Unity or Godot
Backend	Python FastAPI
Memory	Cognee Python SDK
LLM	Start with OpenAI or Gemini for reliability
Embeddings	Use same provider initially
Local graph store	Kuzu default
Local vector store	LanceDB default
Local relational store	SQLite default
Production graph store	Neo4j
Production vector store	PGVector / LanceDB / cloud vector DB
Cache	Redis for sessions
Database for game state	PostgreSQL
Analytics	PostHog / custom event logs
Version control	GitHub
Build target	PC first

Cognee's base install can run with default local databases, and its docs list SQLite, LanceDB, and Kuzu-style local defaults for getting started without external servers. For production or multi-agent use, move away from local file-based graph defaults; Cognee's graph-store docs note Kuzu's concurrency limitation and recommend Neo4j for multi-agent scenarios.

5. Design the NPC memory model

This is the most important part.

You need separate **game state** and **memory state**.

5.1 Game state

Stored in your normal game database / save file.

Example:

```
{
  "day": 2,
  "time_block": "afternoon",
  "player_location": "bakery",
  "quest_states": {
    "garden_shed_mystery": "investigating"
  },
  "npc_states": {
    "maya": {
      "location": "bakery",
      "mood": "tense",
      "trust_player": 42,
      "suspicion_player": 10
    }
  }
}
```

5.2 Memory state

Stored in Cognee.

Each important event becomes a memory entry.

Example:

```
{
  "memory_id": "evt_00291",
  "type": "player_action",
  "subject": "player",
  "verb": "promised",
  "object": "help Maya repair the bakery sign",
  "npc_owner": "maya",
  "location": "bakery",
  "day": 1,
  "time_block": "morning",
  "emotional_valence": 0.6,
  "importance": 0.7,
  "privacy": "personal",
  "witnesses": ["maya"],
  "tags": ["promise", "bakery", "relationship"]
}
```

5.3 Memory categories

Use these categories from day one:

Memory type	Example	Used for
Player action	Player helped Maya	Reputation, trust
Dialogue claim	Player said Sam was near the shed	Contradictions, rumours
Promise	Player promised to return by evening	Accountability
Secret	Maya confessed she lost the shed key	Private dialogue
Witnessed event	Omar saw player enter alley	Mystery evidence
Gossip	Jules heard that Maya argued with Sam	Social spread
Relationship memory	Priya distrusts Omar	NPC-to-NPC behaviour
World fact	Shed lock was broken before noon	Investigation logic
Emotional memory	Maya felt betrayed by player	Tone and future dialogue
Resolved memory	Player fulfilled promise	Long-term trust update

6. Design Cognee datasets

Use datasets to keep memories organized.

Cognee's `remember()` is dataset-aware and writes permanent memory into named datasets, while `recall()` can scope retrieval to specific datasets.

Recommended dataset layout

```
neighbourhood_world_lore
  Static facts: map, locations, factions, backstories

npc_maya_memory
  Maya's personal memory

npc_sam_memory
  Sam's personal memory

npc_jules_memory
  Jules's personal memory

shared_neighbourhood_rumours
```

```
Memories that have spread publicly

case_garden_shed
  Evidence, clues, contradiction records

player_profile
  Player tendencies, promises, reputation summaries

system_design_rules
  Dialogue rules, safety constraints, tone guides
```

Why separate NPC datasets?

Because not every NPC should know everything. Separate datasets let you control who can recall what.

For example:

```
Maya recall scope:
- npc_maya_memory
- shared_neighbourhood_rumours
- neighbourhood_world_lore
- player_profile

Sam recall scope:
- npc_sam_memory
- shared_neighbourhood_rumours
- neighbourhood_world_lore
- player_profile
```

This gives each NPC a different view of the same world.

7. Build the core memory operations

You need four internal backend functions that map cleanly to Cognee's workflow.

Cognee's current v1.0 operations are `remember`, `recall`, `improve`, and `forget`; `remember()` stores memory, `recall()` retrieves memory, `improve()` enriches existing graph memory and can bridge session memory into permanent memory, and `forget()` deletes data or memory state.

7.1 `record_memory()`

Called whenever something important happens.

```
async def record_memory(event):
    await cognee.remember(
        data=json.dumps(event),
        dataset_name=event["target_dataset"],
        node_set=[
            event["npc_owner"],
            event["type"],
            event["privacy"]
        ],
        self_improvement=False
    )
```

Use `self_improvement=False` during high-frequency gameplay to avoid expensive graph enrichment on every tiny action. Batch enrichment later.

7.2 `recall_for_npc()`

Called before dialogue, decision-making, quest checks, or gossip.

```
async def recall_for_npc(npc_id, query, datasets):
    results = await cognee.recall(
        query_text=query,
        datasets=datasets,
        only_context=True
    )
    return [r.text for r in results]
```

Use `only_context=True` when you want your own dialogue generator or authored dialogue system to decide what to do with the retrieved memories.

7.3 `nightly\_improve\_memory()`

Run at the end of each in-game day.

```
async def nightly_improve_memory(datasets):
    for dataset in datasets:
        await cognee.improve(dataset=dataset)
```

This is where memories become more useful for future retrieval. Cognee's `improve()` can enrich existing graph memory and, with session IDs, bridge session memory into the permanent graph.

7.4 `forget\_or\_decay\_memory()`

Use for design-controlled forgetting, not random deletion.

```
async def forget_memory(dataset, data_id):
    await cognee.forget(dataset=dataset, data_id=data_id)
```

Cognee's `forget()` supports deleting individual data items, whole datasets, all memory, or memory-only graph/vector state depending on the use case.

For gameplay, you probably do **not** want true deletion most of the time. Use decay, suppression, or "resolved" tags unless a memory must be erased.

8. Create the memory importance system

Do not send every footprint to Cognee. Create a memory scoring layer.

8.1 Memory scoring formula

Every event gets an importance score from 0 to 1.

```
importance =
    plot_relevance * 0.35
+ relationship_impact * 0.25
+ emotional_intensity * 0.20
+ novelty * 0.10
+ witness_count * 0.10
```

8.2 Store only important events permanently

Importance	Treatment
0.0-0.2	Do not store, or keep only in temporary session
0.2-0.5	Store as short-term NPC memory
0.5-0.8	Store as permanent NPC memory
0.8-1.0	Store as permanent memory + shared case/event memory

8.3 Example

Player buys bread from Maya:

```
importance: 0.1
Do not store permanently.
```

Player promises Maya to speak to Sam:

```
importance: 0.65
Store in Maya memory and player profile.
```

Player lies to Maya about Sam's location:

```
importance: 0.85
Store in Maya memory, player profile, and case memory.
```

9. Design NPC personalities

Each NPC needs two things:

1. **Static character sheet**
2. **Dynamic memory-informed state**

9.1 Static character sheet

Example:

```
{
  "npc_id": "maya",
  "name": "Maya D'Souza",
  "age": 34,
  "role": "bakery owner",
  "personality": ["warm", "observant", "stubborn"],
  "speaking_style": "friendly but direct",
  "secrets": [
    "She borrowed the community shed key and lost it."
  ],
  "goals": [
    "Keep the bakery open",
    "Protect her younger brother Sam",
    "Avoid public embarrassment"
  ],
  "default_relationships": {
    "sam": "protective",
    "jules": "friendly rivalry",
    "omar": "distrust"
  }
}
```

Store this in `neighbourhood_world_lore` and the NPC's own dataset.

9.2 Dynamic NPC state

Stored in game state, updated by memory interpretation.

```
{
  "npc_id": "maya",
  "current_mood": "anxious",
  "trust_player": 58,
  "suspicion_player": 22,
  "known_clues": ["lost_key", "sam_seen_near_shed"],
  "active_concerns": ["player_might_tell_jules"],
  "dialogue_temperature": "guarded"
}
```

Do not rely on the LLM to hold numeric state. Keep numbers in your game system. Use Cognee for memory context.

10. Build the dialogue pipeline

Use a hybrid pipeline, not fully freeform generation.

10.1 Dialogue request flow

```
Player talks to NPC
|
Game sends: npc_id, location, quest state, player line
|
Backend recalls relevant memories from Cognee
|
Dialogue planner chooses intent
|
Dialogue generator creates candidate response
|
Safety/style validator checks output
|
Game displays line + choices
```

|
Important facts from exchange are stored as memories

10.2 Dialogue prompt structure

The backend should assemble a structured prompt like:

```
You are writing dialogue for Neighbourhood Echoes.

NPC:
Maya D'Souza, bakery owner.
Warm, observant, stubborn.
Currently anxious.

Scene:
Bakery, Day 2 afternoon.

Player said:
"Did you give Sam the shed key?"

Relevant memories:
1. Maya remembers she lost the shed key on Day 1.
2. Maya remembers the player promised not to embarrass her.
3. Maya heard a rumour that Sam was seen near the shed.

Rules:
- Do not reveal secrets unless trust_player > 60 or pressure_level > 70.
- Keep response under 45 words.
- Offer 2 player choices.
- Tone: guarded but not hostile.
```

10.3 Dialogue output format

Force structured JSON:

```
{
  "npc_line": "why are you asking about Sam? I told you, the key situation is complicated.",
  "emotion": "guarded",
  "revealed_facts": [],
  "new_memories": [
    {
      "type": "dialogue_claim",
      "subject": "player",
      "verb": "asked_about",
      "object": "Sam and the shed key",
      "importance": 0.55
    }
  ],
  "player_choices": [
    "I'm trying to help you.",
    "Someone saw Sam near the shed."
  ]
}
```

11. Build authored dialogue fallbacks

For reliability, create authored templates for important story beats.

Dialogue categories

Category	Use AI?	Notes
Critical mystery reveals	No / mostly authored	Prevent contradictions
Emotional confrontation	Hybrid	Authored beats, AI wording
Casual small talk	Yes	Memory-driven flavour
Quest instructions	Mostly authored	Avoid ambiguity
Rumours	Hybrid	AI can phrase them
NPC barks	Yes	Cheap, varied, memory-based

This avoids the common problem of AI dialogue breaking quest logic.

12. Build the gossip system

The gossip system is where the game becomes interesting.

12.1 Gossip eligibility

A memory can spread if:

```
privacy != "secret"
AND importance > 0.45
AND emotional_intensity > 0.3
AND source_npc has social connection to target_npc
```

12.2 Gossip distortion

When a memory spreads, create a derived memory.

Original:

```
{
  "fact": "Player found the shed lock broken before noon.",
  "confidence": 0.9
}
```

Gossip version:

```
{
  "fact": "Player was near the shed before everyone else.",
  "confidence": 0.55,
  "distortion": "suspicion_bias",
  "source": "jules"
}
```

12.3 Gossip graph

Each night:

1. Get important memories from NPCs.
2. Check social connections.
3. Create gossip candidates.
4. Apply distortion based on NPC personality.
5. Store gossip in receiving NPC dataset.
6. Store public rumours in shared_neighbourhood_rumours.

Example:

```
Maya tells Jules:
"The player keeps asking about the shed key."

Jules remembers:
"Maya seemed worried that the player knows something about the shed key."
```

This makes information mutate naturally.

13. Build the relationship system

Relationships should be numeric and deterministic, but memory-informed.

13.1 Core relationship variables

For each NPC-to-player relationship:

```
{
  "trust": 50,
  "affection": 40,
  "suspicion": 10,
  "fear": 0,
  "respect": 30,
  "resentment": 5
}
```

For NPC-to-NPC:

```
{
  "maya->sam": {
    "trust": 75,
    "protectiveness": 90,
    "resentment": 20
  }
}
```

13.2 Update rules

Examples:

Event	Effect
Player fulfils promise	+trust
Player breaks promise	-trust, +resentment
Player lies and is caught	-trust, +suspicion
Player protects secret	+trust
Player exposes secret publicly	-trust, +fear or resentment
Player helps disliked NPC	Depends on observer
Player repeats gossip	+bond with gossip-loving NPC, -respect with honest NPC

13.3 Memory-informed relationship update

After a major event:

```
Retrieve:
- NPC's memories of player promises
- NPC's values
- Recent witnessed actions
- Relevant gossip

Then calculate:
- Did this confirm or contradict prior belief?
- Did this help or harm NPC goals?
- Should relationship variables change?
```

Use the LLM to classify the emotional interpretation, but keep the final numeric update rule-based.

14. Build the mystery system

The first mystery should be small but socially tangled.

14.1 Main mystery

The community garden shed was broken into.

Important truth:

```
Maya lost the shed key.
Sam found it.
Omar saw Sam near the shed.
Jules spread a rumour that the player was involved.
```

Priya knows the lock was already damaged before Sam arrived.
The real culprit is not malicious: a child hid inside the shed and broke the latch escaping.

14.2 Mystery design goals

The player should solve by comparing memories:

Clue type	Example
Direct evidence	Broken latch
Witness memory	Omar saw Sam
Contradiction	Sam says he was home; Maya remembers he came to bakery
Secret	Maya lost key
Rumour	Player was near shed
Emotional clue	Maya overreacts when key is mentioned

14.3 Cognee use

For each interrogation, recall memories like:

"What does Maya know about the shed key, Sam, and the player?"

For contradiction detection:

"List memories that contradict Sam's claim that he was home on Day 1 afternoon."

Cognee's recall supports graph-backed retrieval, session-aware retrieval, auto-routing, and source tagging, which is useful when you need to know whether context came from session memory, graph memory, or another source.

15. Design NPC daily simulation

NPCs should not only react when spoken to. They should have routines.

15.1 Daily schedule

Example:

```
{
  "maya": {
    "morning": "bakery",
    "afternoon": "market",
    "evening": "bakery_back_room"
  },
  "sam": {
    "morning": "school_lane",
    "afternoon": "garden",
    "evening": "arcade"
  }
}
```

15.2 Daily simulation tick

At each time block:

1. Move NPCs.
2. Trigger scheduled events.
3. Check NPC goals.
4. Generate NPC-to-NPC interactions.
5. Store important interactions as memories.

6. Spread gossip.
7. Update relationship state.

15.3 Example autonomous event

```
{
  "event_type": "npc_conversation",
  "participants": ["maya", "jules"],
  "location": "market",
  "summary": "Maya asked Jules whether the player had been asking about Sam.",
  "importance": 0.62,
  "memories_created": [
    "maya_remembers_she_warned_jules",
    "jules_remembers_maya_is_nervous"
  ]
}
```

16. Create the backend API

Build a thin backend that exposes game-friendly endpoints.

16.1 Required endpoints

```
POST /memory/record
POST /memory/recall
POST /dialogue/respond
POST /npc/tick
POST /gossip/spread
POST /relationship/update
POST /day/end
GET /debug/npc/{npc_id}/memories
GET /debug/case/{case_id}/timeline
POST /debug/reset
```

16.2 Example `POST /dialogue/respond`

Request:

```
{
  "player_id": "player_001",
  "npc_id": "maya",
  "location": "bakery",
  "day": 2,
  "time_block": "afternoon",
  "player_input": "Did Sam have the shed key?",
  "active_quest": "garden_shed_mystery"
}
```

Response:

```
{
  "npc_line": "Why would Sam have it? Unless... no, never mind.",
  "emotion": "nervous",
  "player_choices": [
    {
      "id": "press_sam",
      "text": "You sound like you know something."
    },
    {
      "id": "reassure",
      "text": "I'm not trying to blame him."
    }
  ],
  "relationship_delta": {
    "trust": -2,
    "suspicion": 5
  },
  "new_memory_ids": ["evt_00392"]
}
```

17. Create the memory schema

Use a strict schema for all memories.

17.1 Base memory object

```
{
  "id": "evt_000001",
  "type": "promise | action | claim | secret | gossip | clue | emotion | relationship",
  "canonical_text": "Player promised Maya they would not tell Jules about the lost key.",
  "subject": "player",
  "verb": "promised",
  "object": "not tell Jules about the lost key",
  "owner_npc": "maya",
  "known_by": ["maya"],
  "source": "direct_conversation",
  "source_confidence": 0.95,
  "truth_status": "true | false | uncertain | rumour | subjective",
  "privacy": "public | personal | secret",
  "importance": 0.72,
  "emotional_valence": 0.4,
  "emotional_tags": ["relieved", "vulnerable"],
  "location": "bakery",
  "day": 1,
  "time_block": "evening",
  "related_quest": "garden_shed_mystery",
  "decay_rate": 0.1,
  "expires_after_days": null,
  "resolved": false,
  "created_at_game_time": "day_1_evening"
}
```

17.2 Relationship memory

```
{
  "type": "relationship",
  "canonical_text": "Maya trusts the player more because they protected her secret.",
  "subject": "maya",
  "verb": "trusts",
  "object": "player",
  "relationship_delta": {
    "trust": 12,
    "resentment": -4
  },
  "reason": "Player did not tell Jules that Maya lost the shed key."
}
```

17.3 Gossip memory

```
{
  "type": "gossip",
  "canonical_text": "Jules heard that the player was asking too many questions about Sam.",
  "original_memory_id": "evt_00301",
  "distortion_level": 0.35,
  "source_npc": "maya",
  "receiver_npc": "jules",
  "truth_status": "subjective"
}
```

18. Implement memory retrieval recipes

Do not use one generic recall query for everything. Create retrieval recipes.

18.1 Dialogue recall

Query:
"What does {npc} remember about the player, this location, the active quest, and the last thing the player said?"

Datasets:

- npc_{npc_id}_memory
- shared_neighbourhood_rumours
- neighbourhood_world_lore
- player_profile

18.2 Suspicion recall

Query:
"What memories make {npc} suspicious of the player or of {target_npc}?"

18.3 Promise recall

Query:
"What promises has the player made to {npc}, and were they fulfilled?"

18.4 Mystery contradiction recall

Query:
"What memories contradict the claim: '{claim}'?"

18.5 Gossip recall

Query:
"What recent important memories would {npc} be likely to share with {target_npc}?"

18.6 Emotional recall

Query:
"What emotional memories affect how {npc} feels about the player right now?"

19. Build the first vertical slice

Phase 1: Prototype memory only

Goal: prove that NPC memory changes dialogue.

Build:

1. CLI or simple web interface.
2. 3 NPCs: Maya, Sam, Jules.
3. Store 10 fixed memories.
4. Ask each NPC the same question.
5. Confirm each gives a different answer based on their memory dataset.

Example test:

Question: "Who do you think broke into the shed?"

Maya: protects Sam.
Sam: denies involvement.
Jules: repeats rumour.

Deliverable:

A backend script where NPC responses differ because of Cognee recall.

Phase 2: Game-engine walking prototype

Goal: let player walk around and trigger memory-backed conversations.

Build:

1. Small map.

2. Player controller.
3. Interaction button.
4. Dialogue UI.
5. Backend API call.
6. Hardcoded NPC locations.
7. Store conversation memories.

Deliverable:

Player can speak to Maya, say something important, leave, return, and Maya remembers it.

Phase 3: Promise system

Goal: make NPCs hold player accountable.

Build:

1. Detect promise-like choices.
2. Store promise memory.
3. Track promise deadline.
4. Check fulfilment.
5. Update relationship.
6. Change dialogue if promise broken.

Example:

Day 1 morning:
Player promises to bring Maya repair tools.

Day 1 evening:
Player returns without tools.

Maya:
"You said you'd help before closing. I waited."

Deliverable:

At least 3 promises with fulfilment/broken outcomes.

Phase 4: Mystery clue system

Goal: make memories useful for investigation.

Build:

1. Evidence objects.
2. Witness statements.
3. NPC claims.
4. Contradiction detection.
5. Case notebook UI.
6. Player accusation flow.

Deliverable:

Player can solve the garden shed mystery using NPC memories and clues.

Phase 5: Gossip system

Goal: make information travel.

Build:

1. NPC social graph.
2. Gossip eligibility rules.
3. Gossip distortion.
4. Nightly gossip spread.
5. NPC dialogue uses received gossip.

Deliverable:

If player tells Maya a secret, Jules may hear a distorted version later depending on Maya/Jules relationship.

Phase 6: Relationship-driven branching

Goal: make social history change available options.

Build:

1. Trust gates.
2. Suspicion gates.
3. Secret reveal thresholds.
4. Relationship memory summaries.
5. Alternative quest paths.

Example:

If Maya trust > 60:
She admits she lost the key.

If Maya trust < 30:
She lies and points suspicion toward Omar.

Deliverable:

Same mystery has at least 3 different solution paths.

20. Build debugging tools early

Memory games are hard to debug. Build tools before adding lots of content.

20.1 NPC memory viewer

For each NPC, show:

Known memories
Memory importance
Source
Truth status
Who told them
When learned
Related quest
Whether resolved

20.2 Conversation trace viewer

Show:

Player input
Recall query
Retrieved memories
Prompt sent to generator
NPC response
New memories created
Relationship changes

20.3 Gossip inspector

Show:

```
Original memory
Source NPC
Target NPC
Distortion
Resulting gossip
Reason it spread
```

20.4 Case timeline

Show all known facts in chronological order:

```
Day 1 morning: Maya loses key.
Day 1 afternoon: Sam finds key.
Day 1 evening: Shed latch breaks.
Day 2 morning: Jules starts rumour.
```

Without these tools, you will not know whether a bad NPC response came from bad memory storage, bad retrieval, bad prompting, or bad game-state logic.

21. Handle latency and cost

21.1 Use memory tiers

Tier	Storage	Use
Immediate	In-game RAM	Current dialogue turn
Session	Cognee session memory / cache	Current conversation or day
Permanent	Cognee graph memory	Long-term NPC memory
Summary	Daily NPC summary	Fast recall
Archive	Low-priority memories	Rare retrieval

Cognee supports session memory via `session_id`, where session memory is the fast path and permanent memory is heavier because it runs ingestion, chunking, graph extraction, embedding, and enrichment.

21.2 Cache common recalls

Cache these for 1-5 minutes of real time:

```
NPC opinion of player
NPC active secrets
NPC current quest memories
Recent rumours
Location-specific memories
```

21.3 Batch writes

During active gameplay:

```
Collect events -> score importance -> store important events -> batch improve at day end
```

Do not run expensive enrichment after every minor line of dialogue.

22. Memory decay and summarisation

A good NPC memory engine needs forgetting, compression, and reinterpretation.

22.1 Daily summary

At day end, create a summary memory per NPC.

Example:

```
Maya's Day 2 Summary:
Maya grew more anxious after the player asked about Sam and the shed key. She still trusts the
player slightly because they did not reveal her lost-key secret to Jules.
```

Store this as a high-importance memory.

22.2 Decay rules

Memory	Decay
Casual greeting	Fast
Minor favour	Medium
Broken promise	Slow
Secret	Very slow
Mystery clue	No decay until case resolved
Emotional betrayal	Slow
Resolved clue	Archive after case closes

22.3 Contradiction handling

Do not delete contradictions. Mark them.

```
{
  "memory_a": "Sam said he was home.",
  "memory_b": "Omar saw Sam near the shed.",
  "relationship": "contradicts",
  "confidence": 0.8
}
```

Contradictions are gameplay fuel.

23. Content production plan

23.1 Write NPC bibles

For each major NPC:

```
Name
Role
Daily routine
Public personality
Private fear
Secret
Relationship to each NPC
Speech style
Memory biases
What they notice
What they ignore
What they gossip about
What they never share
```

23.2 Write memory seeds

Each NPC starts with 20-40 memories.

Example for Maya:

```
Maya remembers Sam used to hide in the garden shed as a child.
Maya remembers Jules exaggerates stories when bored.
Maya remembers Omar once accused Sam unfairly.
Maya remembers she borrowed the shed key and misplaced it.
```

23.3 Write quest beats

For each quest:

```
Required facts
Optional facts
NPCs who know each fact
When they learned it
What they believe incorrectly
What changes their mind
```

23.4 Write response constraints

For each secret:

```
Secret: Maya lost the shed key.
Reveal if:
- trust_player > 60
OR player has evidence "bakery_key_tag"
OR pressure_level > 80

Deny if:
- suspicion_player > 50

Deflect if:
- Jules is present
```

24. Build the AI safety and consistency layer

Even in a cozy game, AI output must be controlled.

24.1 Response validator

Reject or regenerate dialogue if it:

```
Reveals locked facts
Contradicts known game state
Mentions unavailable locations
Invents new NPCs
Changes the culprit
Breaks tone
Is too long
Contains unsafe content
```

24.2 Canonical fact checker

Before displaying dialogue:

1. Extract factual claims from response.
2. Compare against game state and case state.
3. Allow subjective claims.
4. Block impossible claims.

Example:

```
NPC says: "I saw Sam at midnight."
But game has no midnight time block.
Reject or rewrite.
```

24.3 Memory write validator

Before storing a new memory:

```
Is this memory important?
Who should know it?
Is it a fact, claim, lie, rumour, or emotion?
Does it duplicate an existing memory?
Should it update an existing memory instead?
```

25. Save/load strategy

You need to save both deterministic state and memory state.

25.1 Save file contains

```
{
  "save_id": "slot_01",
  "game_state": {},
  "relationship_state": {},
  "quest_state": {},
  "cognee_dataset_ids": {
    "maya": "npc_maya_memory_slot_01",
    "sam": "npc_sam_memory_slot_01"
  },
  "memory_version": "0.1.0"
}
```

25.2 Dataset naming convention

```
save_{save_id}_npc_{npc_id}_memory
save_{save_id}_shared_rumours
save_{save_id}_case_{case_id}
save_{save_id}_player_profile
```

25.3 New game reset

Use Cognee's dataset deletion or `forget()` carefully for new game cleanup. Cognee supports dataset-level deletion and memory-only reset, but you should avoid deleting global world-lore datasets shared across saves.

26. Testing plan

26.1 Unit tests

Test:

```
Memory importance scoring
Dataset routing
Promise detection
Relationship deltas
Gossip eligibility
Gossip distortion
Secret reveal gates
Contradiction detection
```

26.2 Simulation tests

Run automated 3-day simulations.

Example:

```
Simulation A:
Player helps Maya, ignores Sam, tells Jules everything.

Simulation B:
Player protects Maya, confronts Sam, avoids gossip.

Simulation C:
Player lies to everyone.
```

Check:

Do NPCs remember correctly?
Do relationships change plausibly?
Does the mystery remain solvable?
Do NPCs reveal facts too early?
Does gossip become nonsense?

26.3 Golden path tests

Author fixed expected outcomes.

Example:

Given:
Player promised Maya secrecy.
Player told Jules the secret.

Expected:
Maya trust decreases.
Maya confronts player.
Jules has gossip memory.
Shared rumour may appear next day.

26.4 Player testing

Watch for:

Do players understand NPCs are remembering?
Do they blame the system when NPCs react?
Are consequences too subtle?
Are consequences too punishing?
Do players feel they can repair relationships?

27. Production milestones

Milestone 0: Design lock

Deliverables:

Game design one-pager
NPC memory schema
First mystery outline
NPC list
Technical architecture

Exit criteria:

Everyone can explain the core loop in one sentence.

Milestone 1: Cognee memory spike

Deliverables:

Python backend
Cognee installed
3 NPC datasets
remember/recall working
memory debug viewer

Exit criteria:

Same player question produces different NPC answers based on different memories.

Milestone 2: Dialogue vertical slice

Deliverables:

Game scene
1 location
2 NPCs
Dialogue UI
Backend dialogue endpoint
Memory storage after conversation

Exit criteria:

NPC remembers something said earlier in the session.

Milestone 3: Persistent NPC memory

Deliverables:

Save/load
Dataset-per-save naming
Permanent memory writes
Day-end memory improvement
NPC daily summaries

Exit criteria:

NPC remembers a Day 1 event on Day 3 after reloading the save.

Milestone 4: Promise and consequence system

Deliverables:

Promise detection
Promise deadlines
Fulfil/break logic
Relationship updates
Dialogue consequences

Exit criteria:

Breaking a promise changes future dialogue and relationship state.

Milestone 5: Mystery gameplay

Deliverables:

Garden shed mystery
Evidence objects
Witness memories
Case notebook
Contradiction UI
Accusation flow

Exit criteria:

Player can solve or mishandle the mystery using remembered NPC statements.

Milestone 6: Gossip simulation

Deliverables:

NPC social graph
Gossip spread
Gossip distortion
Shared rumour memory
Rumour-based dialogue

Exit criteria:

Information told to one NPC can affect another NPC later.

Milestone 7: Full MVP polish

Deliverables:

6 major NPCs
3 in-game days
1 complete mystery
3 side stories
Debug tools
Performance budget
Playable build

Exit criteria:

A 45-90 minute playable slice with multiple endings and visible memory-driven consequences.

28. Recommended folder structure

```
neighbourhood-echoes/  
  game-client/  
    Assets/  
    Scripts/  
      Dialogue/  
      NPC/  
      Quest/  
      SaveSystem/  
      UI/  
  backend/  
    app/  
      main.py  
      api/  
        dialogue.py  
        memory.py  
        gossip.py  
        npc.py  
        debug.py  
      cognee_layer/  
        client.py  
        datasets.py  
        recall_recipes.py  
        memory_writer.py  
      simulation/  
        schedules.py  
        gossip_engine.py  
        relationship_engine.py  
        day_end.py  
      dialogue/  
        planner.py  
        generator.py  
        validator.py  
      schemas/  
        memory.py  
        npc.py  
        dialogue.py  
        quest.py  
      tests/  
  content/  
    npcs/  
      maya.json  
      sam.json  
      jules.json  
    quests/  
      garden_shed_mystery.json  
    world/  
      locations.json  
      lore.md  
  tools/  
    memory_viewer/  
    simulation_runner/
```

29. First implementation order

Build in this exact order:

1. Create content JSON for 3 NPCs.
2. Create backend memory schema.
3. Implement Cognition `remember()` wrapper.
4. Implement Cognition `recall()` wrapper.
5. Seed NPC memories.
6. Build CLI dialogue test.
7. Add game client dialogue UI.
8. Connect game client to backend.
9. Store player dialogue as memory.
10. Add relationship variables.
11. Add promise detection.
12. Add day-end summary.
13. Add gossip spread.
14. Add mystery notebook.
15. Add debug memory viewer.
16. Expand from 3 NPCs to 6 NPCs.

This order keeps risk low. You prove memory before building too much game around it.

30. MVP feature checklist

Must have

NPCs remember player dialogue
NPCs remember player actions
NPCs remember promises
NPCs react differently based on trust/suspicion
NPCs can hold secrets
NPCs can hear rumours
Player can inspect clues
Mystery can be solved
Save/load preserves memory
Debug viewer exists

Should have

Daily NPC summaries
Memory decay
Gossip distortion
Contradiction detection
NPC-to-NPC relationship changes
Multiple endings

Could have later

Voice synthesis
Fully simulated economy
Procedural side quests
Generated neighbourhood expansions
Multiplayer
Mobile version
Modding tools

31. Key design risks

Risk 1: NPCs say too much

Solution:

```
Use secret reveal gates.  
Validate every generated response.  
Keep critical reveals authored.
```

Risk 2: Memory retrieval is noisy

Solution:

```
Use strict memory schemas.  
Use dataset scoping.  
Use specific recall recipes.  
Store daily summaries.  
Tune importance scoring.
```

Risk 3: Costs become high

Solution:

```
Batch memory writes.  
Use session memory for short-term interactions.  
Run improve at day end.  
Cache recurring NPC context.  
Use smaller models for extraction.
```

Cognee's docs explicitly distinguish session memory as the fast path and permanent memory as the heavier path involving ingestion, chunking, graph extraction, embeddings, and enrichment.

Risk 4: The game becomes unpredictable

Solution:

```
Keep quest state deterministic.  
Use AI for expression and interpretation, not final authority.  
Make important story facts canonical.
```

Risk 5: Players do not notice memory

Solution:

```
Make NPCs explicitly refer to past events.  
Add UI hints like "Maya remembers your promise."  
Let players repair damaged trust.  
Show rumours spreading through dialogue.
```

32. Final target experience

A good first public demo should produce moments like:

```
Day 1:  
Player promises Maya not to tell anyone she lost the shed key.  
  
Day 2:  
Player tells Jules.  
  
Day 3:  
Maya refuses to help the player and says:  
"I trusted you with one thing. By breakfast, Jules was smiling at me like she knew."  
  
Meanwhile, Sam trusts the player more because exposing Maya shifted suspicion away from him.  
  
The mystery remains solvable, but the social path has changed.
```

That is the heart of **Neighbourhood Echoes**: not just NPCs with dialogue, but NPCs with social memory.

References

Cognee documentation referenced in this plan:

- Core concepts overview: <https://docs.cognee.ai/core-concepts/overview>
- Python API: <https://docs.cognee.ai/python-api>
- Installation and defaults: <https://docs.cognee.ai/getting-started/installation>
- Remember operation: <https://docs.cognee.ai/core-concepts/main-operations/remember>
- Recall operation: <https://docs.cognee.ai/core-concepts/main-operations/recall>
- Improve operation: <https://docs.cognee.ai/core-concepts/main-operations/improve>
- Forget operation: <https://docs.cognee.ai/core-concepts/main-operations/forget>